
Rapport IA02

Daria EZHOVA - Nina COIFFIN - Pierre RIBET

May 2026

Contents

Partie I	2
Présentation des données	2
Contexte et origine	2
Contenu du dataset	2
Exemples d'images	3
Conversion en niveaux de gris	3
Motivation de l'utilisation	4
Importation en python	4
Entraînement des modèles	5
Algorithmes de machine learning	5
CNN	5
CNN Hybride	5
Comparaison des résultats	5
Partie II	6
Présentation des données	6
Chargement des données	6
Visualisation des données	7
Nettoyage des données	9
Pipeline d'entraînement	9
Modèles	11
CNN	11
CNN BiLSTM	12
CNN 2D & 1D	13
Comparaisons des résultats	14
Augmentation des données	15
Résultats / Impact	15
Nouvelles données	16
Nos échantillons	16
Bonus : GAN	17
Bonus : Reconnaissance des locuteurs	17
Conclusion	18
Annexes	19
Bibliographie	34

Partie I

Présentation des données

Le dataset CIFAR-10 est utilisé dans la recherche en vision par ordinateur, pour les tâches de classification d'images. Il permet d'évaluer la performance de différents modèles d'apprentissage automatique, notamment les réseaux de neurones convolutifs (CNN).

Contexte et origine

Le dataset a été développé en 2009, par *Alex Krizhevsky, Vinod Nair et Geoffrey Hinton* dans le cadre de travaux sur l'apprentissage profond et la reconnaissance d'images. Il est disponible sur le site officiel CIFAR Dataset il a été utilisé dans des concours et benchmarks internationaux en vision par ordinateur.

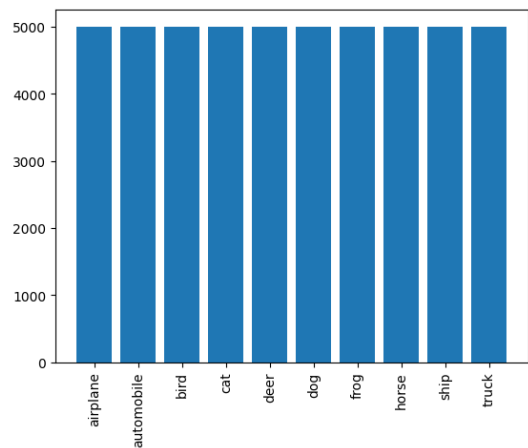
Son objectif principal est de fournir un jeu de données suffisamment complexe pour tester des modèles de classification tout en restant accessible pour l'entraînement rapide, grâce à la taille modérée des images.

Contenu du dataset

Le CIFAR-10 contient 60 000 images en couleur, de dimension 32x32 pixels, réparties en 10 classes distinctes représentant différents objets et animaux.

Classe (anglais)	Classe (français)
Airplane	Avion
Automobile	Voiture
Bird	Oiseau
Cat	Chat
Deer	Cerf
Dog	Chien
Frog	Grenouille
Horse	Cheval
Ship	Bateau
Truck	Camion

Correspondance des classes du dataset CIFAR-10 en anglais et en français.



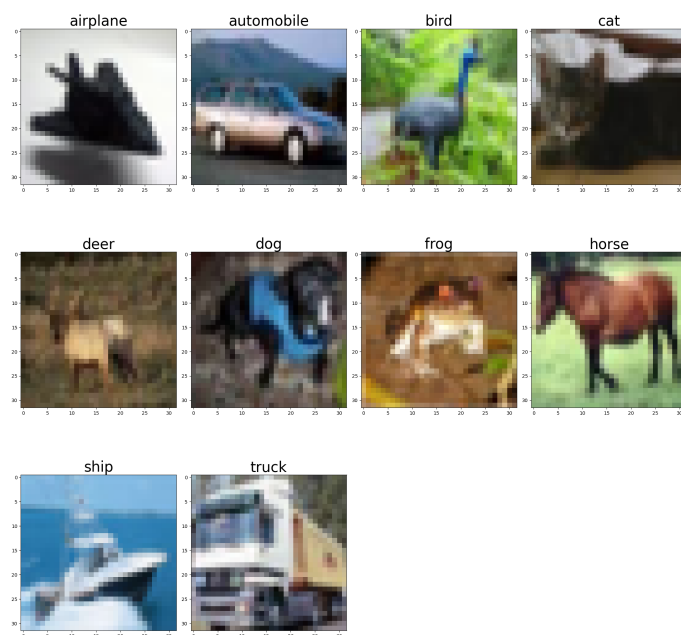
Répartition équilibrée des classes dans le dataset CIFAR-10.

Une vérification rapide de la répartition des classes montre qu'aucune catégorie n'est déséquilibrée en nombre d'échantillons (cf. Figure).

Les concepteurs du dataset ont pris soin à ce que Voiture et Camion, des classes proches visuellement dans notre imaginaire, sont considérées comme strictement distinctes (*Les voitures n'incluent pas de véhicules pouvant être confondu avec des camions*), ce qui évite la confusion pour les modèles de classification.

Exemples d'images

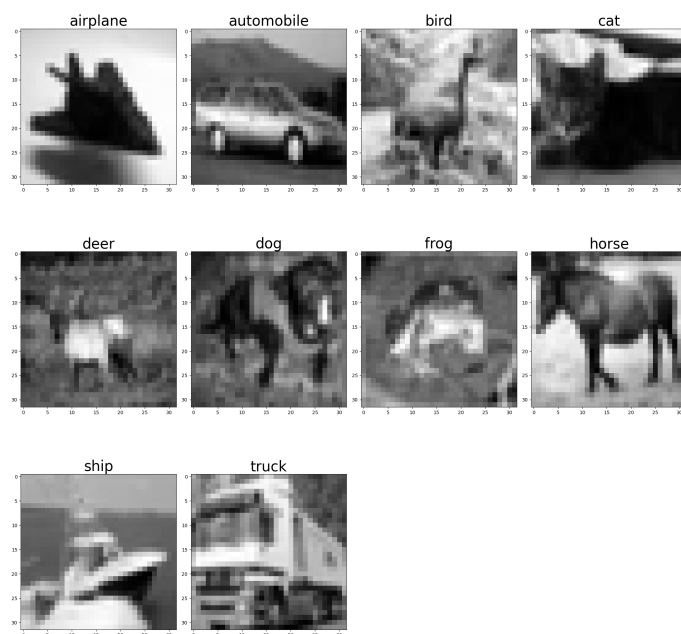
La Figure illustre un exemple représentatif des images de chaque classe. Ces images, malgré leur petite taille, contiennent suffisamment d'informations visuelles pour permettre la différenciation entre les catégories.



Exemple d'images fournies pour chaque classe du dataset CIFAR-10.

Conversion en niveaux de gris

Pour simplifier l'entrée de nos modèles et réduire le coût computationnel, nous avons choisi de convertir les images couleur (RGB) en images en niveaux de gris. Chaque image, initialement composée de trois canaux (rouge, vert, bleu), sera donc représentée par un seul canal de *luminance*. (cf. Figure).



Exemple d'images du dataset CIFAR-10 converties en niveaux de gris.

Motivation de l'utilisation

L'étude de ce dataset permet de :

- Tester et comparer des modèles de classification standard et avancés.
- Explorer l'impact de la simplification des données (conversion en niveaux de gris) sur les performances.
- Développer des pipelines d'apprentissage automatique robustes, applicables ensuite à des images plus grandes ou à des applications réelles.

Le CIFAR-10 est donc un excellent terrain de test pour la classification d'images, d'autant plus qu'il n'est qu'un sous-ensemble d'un dataset labélisé de 80 millions d'images (Tiny Images). Il permet ainsi de faire un premier pas dans le domaine de la vision par ordinateur.

Importation en python

Listing 1: Importation du dataset

```
1 import torchvision
2 # Téléchargement du dataset CIFAR-10
3 data = torchvision.datasets.CIFAR10(download=True, root="./")
4 # Définition des labels des classes
5 # ATTENTION : L'ordre est important
6 labels = ["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck"]
```

Listing 2: Conversion en niveau de gris

```
1 from PIL import Image
2 img = Image.open("image.png")
3 img_gray = img.convert("L") # Conversion en niveaux de gris
4 img_gray.save("image_gris.png")
```

Listing 3: Usage du dataset chargé

```
1 def grayscale_collate_fn(batch):
2     images, labels = zip(*batch)
3     images_gray = [img.convert("L") for img in images]
4
5     images_tensor = [torch.tensor(img, dtype=torch.float32) / 255.0 for img in images_gray]
6     labels_tensor = torch.tensor(labels, dtype=torch.long)
7     return torch.stack(images_tensor), labels_tensor
8
9 train_loader = DataLoader(
10     data,
11     batch_size=32,
12     shuffle=True,
13     collate_fn=grayscale_collate_fn
14 )
```

Entrainement des modèles

Algorithmes de machine learning

Proposition 1

Proposition 2

CNN

Architecture 1

Architecture 2

Architecture 3

CNN Hybride

Comparaison des résultats

Performances en exactitude et temps de calcul

Critères d'évaluation :

- Analyse du dataset
- Choix des algorithmes
- Choix des hyperparamètres
- Performances en exactitude / temps de calcul

Partie II

Présentation des données

La reconnaissance automatique de la parole est un domaine majeur de l'intelligence artificielle visant à convertir un signal audio en information exploitable. Dans ce projet, nous nous intéressons à une tâche simplifiée appelée *keyword spotting*, qui consiste à reconnaître des mots courts prédéfinis.

Le dataset Speech Commands est une base de données audio dédiée à la reconnaissance vocale de mots-clés. Il provient d'un projet de Google, il contient des milliers d'enregistrements de courtes commandes vocales prononcées par différents locuteurs. Chaque échantillon audio est associé à plusieurs informations : le signal audio, la fréquence d'échantillonnage, le label correspondant au mot prononcé, l'identifiant du locuteur et le numéro d'énoncé. Le dataset est utilisé pour entraîner et évaluer des modèles de reconnaissance vocale léger. Dans cette étude nous proposons de nous concentrer sur la reconnaissance du vocabulaire, plutôt que celle du locuteur.

Chargement des données

Listing 4: Chargement du dataset d'entraînement Speech Commands

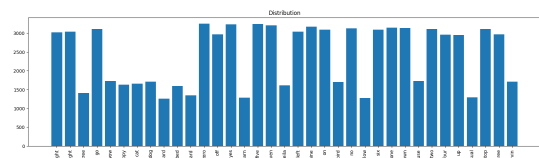
```
1 import torchaudio
2 train = torchaudio.datasets.SPEECHCOMMANDS(download=True, root=".", subset="training")
```

Le dataset fournit par torch est divisé en trois sous-ensemble : Training, Validation, Testing

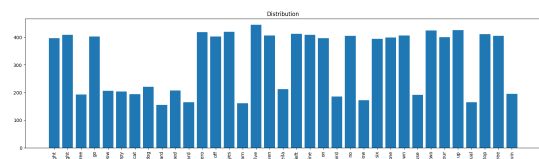
- Nombre d'échantillon d'entraînement : 84843
- Nombre d'échantillon de validation : 9981
- Nombre d'échantillon de test : 11005

Label
right
eight
tree
go
wow
happy
cat
dog
forward
bed
backward
zero
off
yes
learn
five
seven
sheila
left
nine
on
bird
no
follow
six
one
down
house
two
four
up
visual
stop
three
marvin

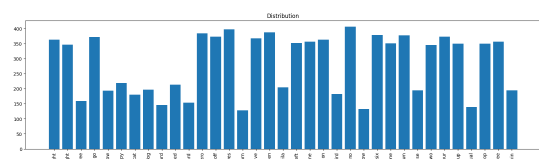
Liste des 35 labels du dataset



Distribution entraînement



Distribution test



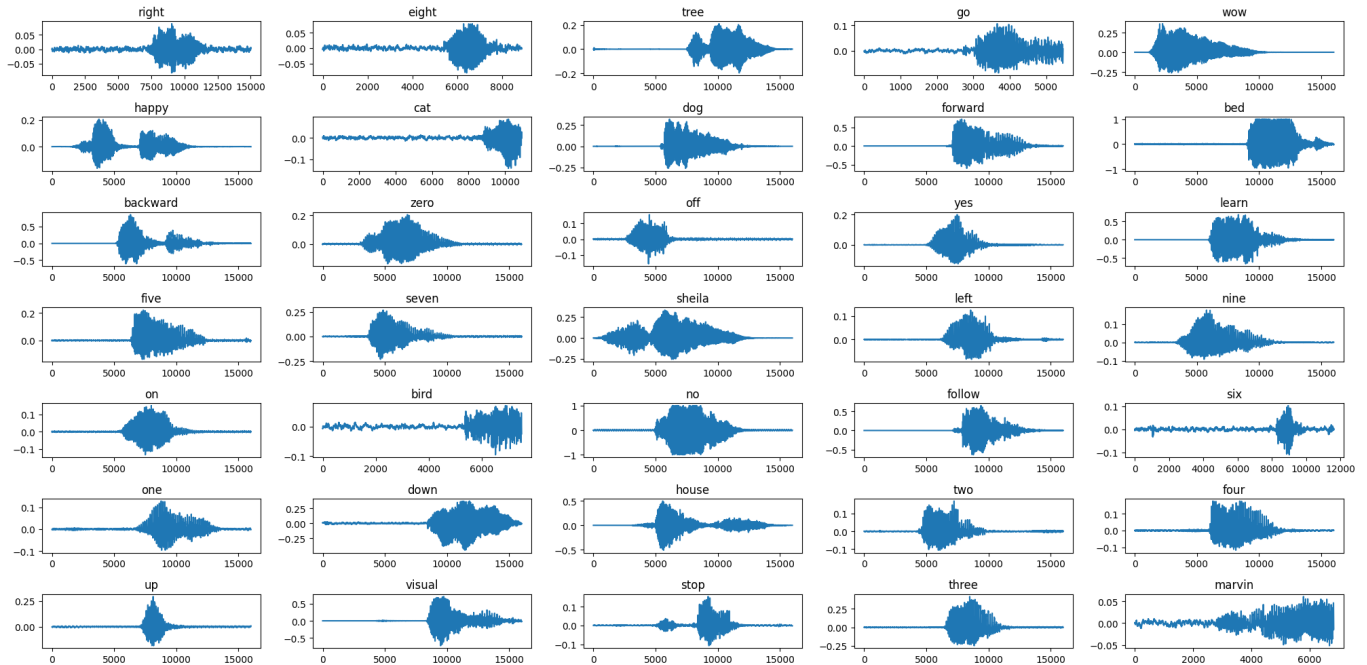
Distribution validation

On remarque que les distribution entre les sous-ensembles sont équivalentes.

Visualisation des données

Waveforms

Le signal audio brut peut être représenté sous la forme d'une waveform (forme d'onde), c'est-à-dire une représentation de l'amplitude du signal en fonction du temps. Chaque point du signal correspond à une mesure capturée à un instant précis lors de l'enregistrement. Dans le cas du dataset, les enregistrements sont échantillonnés à une fréquence de 16kHz, ce qui signifie que 16 000 valeurs sont mesurées par seconde. Les waveforms permettent d'observer certaines caractéristiques globales du signal, comme l'intensité de la voix, la durée de la prononciation ou encore les silences présents dans l'audio.



Exemple de waveforms provenant du dataset Speech Commands.

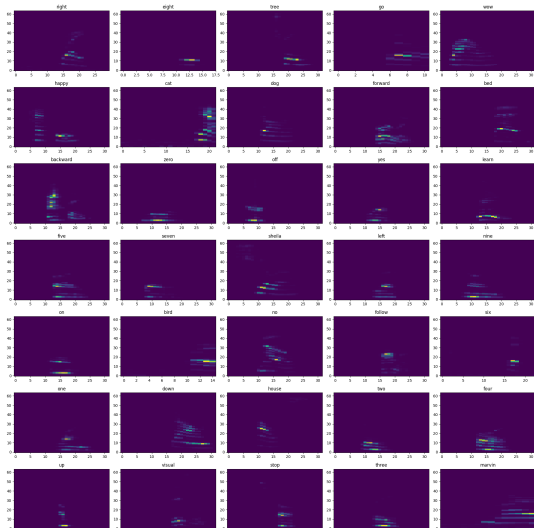
On obtient l'intensité du son à chaque instant, mais cela ne permet pas d'identifier précisément les fréquences présentes dans le signal. Or, dans la parole humaine, les informations essentielles pour différencier les mots sont principalement portées par les variations fréquentielles des sons prononcés. Deux mots peuvent ainsi avoir des waveforms visuellement similaires malgré des contenus fréquentiels très différents. De plus, un même mot peut produire des formes d'onde différentes selon le locuteur, l'intonation ou la vitesse de prononciation. Pour rendre ces caractéristiques plus exploitables par les modèles de classification, il est donc nécessaire de passer à une représentation temps-fréquence.

MelSpectrogrammes

Un spectrogramme est une représentation visuelle des fréquences présentes dans un signal audio au cours du temps. Il est obtenu grâce à une transformation mathématique appelée Short-Time Fourier Transform, qui consiste à découper le signal audio en petites fenêtres temporelles puis à appliquer une transformée de Fourier sur chacune d'elles. Cette opération permet d'observer l'évolution du contenu fréquentiel du signal.

Dans un spectrogramme, l'axe horizontal représente le temps, l'axe vertical les fréquences et l'intensité des couleurs représente l'énergie associée à chaque fréquence. Cette représentation est particulièrement adaptée à la reconnaissance vocale, car elle met en évidence les structures fréquentielles caractéristiques des phonèmes et des mots prononcés.

Cependant, l'oreille humaine ne perçoit pas les fréquences de manière linéaire. Les basses fréquences sont perçues avec beaucoup plus de précision que les hautes fréquences. Afin de reproduire ce comportement, nous utilisons des MelSpectrogrammes. Ceux-ci appliquent une échelle de fréquences appelée *échelle de Mel*, qui compresse les hautes fréquences et accorde davantage d'importance aux basses fréquences. Cette représentation est largement utilisée dans les tâches de reconnaissance vocale car elle fournit des caractéristiques plus proches de la perception humaine.



Exemple de MelSpectrogrammes générés à partir des échantillons audio du dataset Speech Commands.

Listing 5: Fonction qui permet de mettre tous les échantillons a la bonne longueur au sein d'un dataset

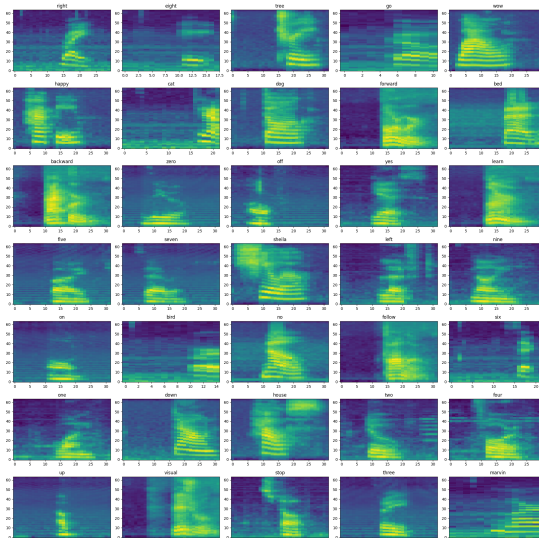
```

1 self.spec = T.MelSpectrogram(
2     n_fft=1024,          # Taille de la fenetre FFT,
                           # plus cette valeur est grande, meilleure est
                           # la resolution en frequence, mais moins bonne
                           # est la resolution temporelle.
3
4     sample_rate=16000, # Frequence d'echantillonnage
                           # du signal audio (en Hz).
5
6     n_mels=64,           # Nombre de bandes de
                           # frequence de Mel, plus ce nombre est eleve,
                           # plus la representation frequentielle est
                           # detaillee.
7
8     hop_length=512       # Nombre d'echantillons entre
                           # deux fenetres FFT consecutives, determine le
                           # chevauchement entre les fenetres et influence
9 )

```

Les Spectrogrammes peuvent être interprétés comme des images. Les CNN peuvent alors détecter des motifs fréquentiels, de la même manière qu'ils détectent des formes ou des textures dans des images classiques.

Enfin, nous appliquons une transformation logarithmique sur les spectrogrammes, car certaines fréquences peuvent avoir des amplitudes très élevées tandis que d'autres possèdent des amplitudes beaucoup plus faibles. Le passage au logarithme permet de réduire cet écart et de rendre les détails fréquentiels plus visibles pour le modèle.



Exemple de Log-MelSpectrogrammes utilisés comme entrée des modèles de classification.

Nettoyage des données

Ici, nous abordons le problème de la variabilité de la durée des échantillons audio au sein du dataset, que nous avons remarquer lors de notre analyse exploratoire (cf. Notebook ExplorationDonnées.ipynb). Bien que tous les signaux soient plus courts que 1 seconde, leurs longueurs diffèrent, ce qui peut poser problème pour les modèles nécessitant des entrées de taille fixe. Pour uniformiser les données, nous avons choisi de compléter les signaux avec des zéros (équivalent à du silence) à la fin de chaque échantillon, afin d'atteindre une durée standard de 1 seconde. Cette approche garantit que tous les signaux ont la même longueur, tout en préservant leur contenu. Les échantillons dont la longueur est inférieure à 16 000 sont donc remplis avec des zéros pour atteindre cette taille cible.

Listing 6: Fonction qui permet de mettre tous les échantillons à la bonne longueur au sein d'un dataset

```

1 def pre_process_batch(batch):
2     target_len = 16000
3     waves, labs = [], []
4
5     for waveform, sr, label_str, _, _ in batch:
6
7         if waveform.dim() == 1:
8             waveform = waveform.unsqueeze(0)
9
10        if waveform.shape[1] < target_len:
11            waveform = F.pad(waveform, (0, target_len - waveform.shape[1]))
12        else:
13            waveform = waveform[:, :target_len]
14
15        label = label_to_idx.get(label_str)
16        if label is None:
17            continue
18
19        waves.append(waveform)
20        labs.append(label)
21
22    if len(waves) == 0:
23        return torch.empty(0), torch.empty(0)
24
25    return torch.stack(waves), torch.tensor(labs)

```

Listing 7: Exemple d'utilisation de la fonction

```

1 from torch.utils.data import DataLoader
2
3 train_loader = DataLoader(
4     data_train,
5     batch_size=32,
6     shuffle=True,
7     collate_fn=pre_process_batch,
8     pin_memory = True,
9     num_workers=8,
10    persistent_workers=True
11 )

```

Pour optimiser le chargement des données et éviter que le CPU ne devienne un goulot d'étranglement pendant l'entraînement, nous utilisons 8 workers. Ces workers sont des processus parallèles qui chargent et préparent les données en arrière-plan, permettant au GPU de se concentrer sur les calculs du modèle. L'option `persistent_workers=True` maintient ces processus actifs entre les itérations, réduisant ainsi le temps de latence lié à leur initialisation. Enfin, `pin_memory=True` permet de transférer les données directement vers la mémoire du GPU, améliorant encore les performances lors de l'entraînement.

Pipeline d'entraînement

Le pipeline d'entraînement suit une approche structurée pour évaluer et optimiser les modèles. Dans un premier temps, chaque modèle est entraîné sur le jeu de données d'entraînement. À la fin de chaque epoch, le modèle est évalué sur le jeu de validation afin de surveiller sa généralisation et d'éviter le overfitting. Cette étape permet de mettre en place l'early stopping.

Enfin, une fois l'entraînement terminé, les modèles sont testés sur le jeu de test, qui n'a jamais été utilisé pendant les phases d'entraînement ou de validation. Cela garantit une évaluation impartiale et permet de comparer objectivement les performances des différents modèles proposés.

Pour plus de détails sur l'implémentation, se référer à l'annexe.

Hyperparameter Tuning

Dans cette section, nous détaillons la méthode utilisée pour optimiser les hyperparamètres des modèles présentés. *Nous ne nous concentrons pas sur l'optimisation des architectures, car en IA01 nous avons déjà proposé une approche pour la recherche d'architectures de réseaux de neurones.*

Processus d'optimisation

Chaque modèle a vu ses hyperparamètres être fine-tuné avec **Optuna** selon les critères suivants :

- **Workers** : 8 pour l'entraînement et 8 pour la validation, afin de paralléliser efficacement les calculs.
- **Matériel** : L'ensemble du traitement est réalisé sur **GPU** pour accélérer les itérations.
- **Paramètres optimisés** : `BatchSize`, `LearningRate` et `WeightDecay` sont recherchés dynamiquement.
- **Nombre d'essais** : 50 tentatives avec Optuna, chaque essai exécutant 3 *epochs* pour évaluer rapidement la performance.

Nous avons utilisé aussi la capacité d'Optuna à utiliser le *pruning*, une technique qui arrête prématurément les essais peu prometteurs, économisant ainsi du temps et des ressources.

Méthodes statiques des modèles

Chaque modèle implémente deux méthodes statiques :

- `optimize` : Génère les paramètres pour un essai Optuna.
- `optimize_defaults` : Attribue des paramètres par défaut au modèle si Optuna n'est pas utilisé. Ces paramètres peuvent être mis à jour ultérieurement pour refléter les meilleurs résultats trouvés par Optuna.

Pour une compréhension approfondie de cette méthode, nous vous invitons à consulter les implémentations des modèles en annexe.

Nous sommes conscients que cette approche ne couvre pas toutes les possibilités d'optimisation. Cependant, elle établit une base solide et démontre notre capacité à concevoir des modèles compatibles avec des outils comme Optuna.

Modèles

CNN

Motivation

Les CNN sont utilisés en traitement du signal audio, notamment pour des tâches comme le Keyword Spotting. Leur capacité à extraire des caractéristiques locales à partir de représentations comme les spectrogrammes, en fait un choix évident pour cette application. En appliquant des filtres convolutifs sur les spectrogrammes, les CNN peuvent identifier des motifs fréquentiels spécifiques aux mots-clés, tout en étant robustes aux variations de prononciation ou de bruit. De plus les CNN offrent un bon compromis entre précision et efficacité, ce qui les rend adaptés à des tâches en temps réel comme la détection de mots-clés.

Définition

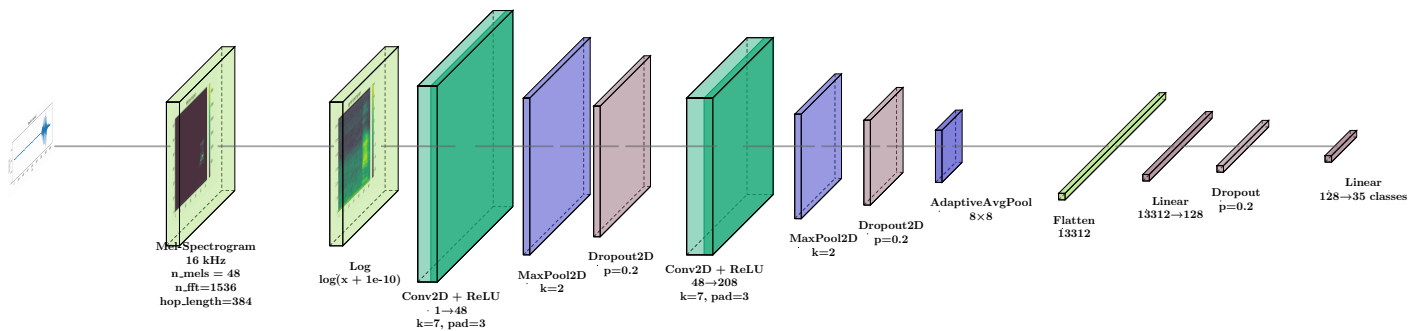
Au départ, nous avons testé une architecture conçue de manière empirique, qui a déjà donné de très bons résultats. *Nous nous sommes rendu compte, après le fine-tuning, qu'il manquait un paramètre optimisable : la taille de la couche intermédiaire (128).*

Résultats optima pour le CNN classique :

Listing 8: Résultat optuna

```
1 Meilleure paramètres : {'batch_size': 16, 'lr': 0.00010012603121343938, 'weight_decay':
  1.6319322258025493e-07, 'n_mels': 48, 'n_fft': 1536, 'hop_length': 384, 'cnn1_out_channels': 48,
  'cnn2_out_channels': 208, 'kernel_size': 7, 'pooling_kernel': 2, 'adaptive_pool_output': (8, 8)
  , 'dropout': 0.2}
2 Meilleure accuracy : 0.8939985973349364
```

On se retrouve donc avec la forme suivante :



Architecture pour le CNN

Pour voir l'implémentation se référer à l'annexe

Pour les modèles CNNTrad et CNNOne, inspirés d'un article de Google, les résultats n'ont pas été à la hauteur de nos attentes, car ces architectures n'étaient pas adaptées à nos données. Néanmoins, nous avons relevé le défi de les implémenter pour explorer leur potentiel. **Papier relatif**

Nous proposons donc d'implémenter les deux architectures proposer dans le papier.

CNNTrad

Listing 9: Exemple d'utilisation de la fonction

```
1 Meilleure paramètres : {'batch_size': 16, 'lr': 0.00021495551281313965, 'weight_decay':
  7.109975589372269e-07, 'dropout': 0.0}
2 Meilleure accuracy : 0.8765654744013626
```

Pour voir l'implémentation se référer à l'annexe et au papier relatif

CNNOne

Listing 10: Exemple d'utilisation de la fonction

```
1 Meilleure paramètres : {'batch_size': 32, 'lr': 0.0006425519426416211, 'weight_decay':
  2.929655627975144e-05, 'dropout': 0.0}
2 Meilleure accuracy : 0.7901011922653041
```

Pour voir l'implémentation se référer à l'annexe et au papier relatif

CNN BiLSTM

Motivation

Les CNN sont efficaces pour extraire des caractéristiques locales à partir de spectrogrammes, mais ils ne capturent pas les dépendances temporelles à long terme dans les signaux audio. Les réseaux récurrents, comme les BiLSTM (Bidirectional Long Short-Term Memory), sont conçus pour modéliser ces dépendances en traitant les séquences dans les deux sens (avant et arrière). En combinant les CNN pour l'extraction de caractéristiques spectrales et les BiLSTM pour la modélisation temporelle, nous proposons une architecture hybride qui pourrait améliorer la reconnaissance des mots-clés. Cette approche nous semble prometteuse, car elle permet de tirer parti des forces des deux types d'algorithme : la capacité des CNN à détecter des motifs locaux dans les spectrogrammes et celle des BiLSTM à capturer le contexte temporel des séquences audio.

Définition

Listing 11: Résultat optimal

```
1 Meilleur parametres : {'batch_size': 64, 'lr': 0.0010061137930597018, 'weight_decay':  
    1.1559805487396425e-06, 'n_mels': 64, 'n_fft': 1024, 'hop_length': 256, 'cnn1_out_channels': 32,  
    'cnn2_out_channels': 96, 'kernel_size': 7, 'pooling_kernel': 2, 'adaptive_pool_output': (8, 8),  
    'lstm_hidden_size': 160, 'lstm_num_layers': 3, 'fc_hidden_size': 384, 'dropout': 0.1}  
2 Meilleur accuracy : 0.9217513275222924
```

CNN 2D & 1D

Motivation

Les CNN 1D peuvent être appliqués directement sur les waveforms (signaux audio bruts) pour extraire des caractéristiques temporelles. En combinant celui-ci avec le CNN 2D, nous cherchons à exploiter à la fois les informations temporelles (1D) et spectrales (2D) pour améliorer la robustesse et la précision du modèle. Cette hybridation devrait permettre de tirer parti des forces de chaque méthode : la capacité des CNN 1D à modéliser les dépendances temporelles locales et celle des CNN 2D à analyser les motifs fréquentiels.

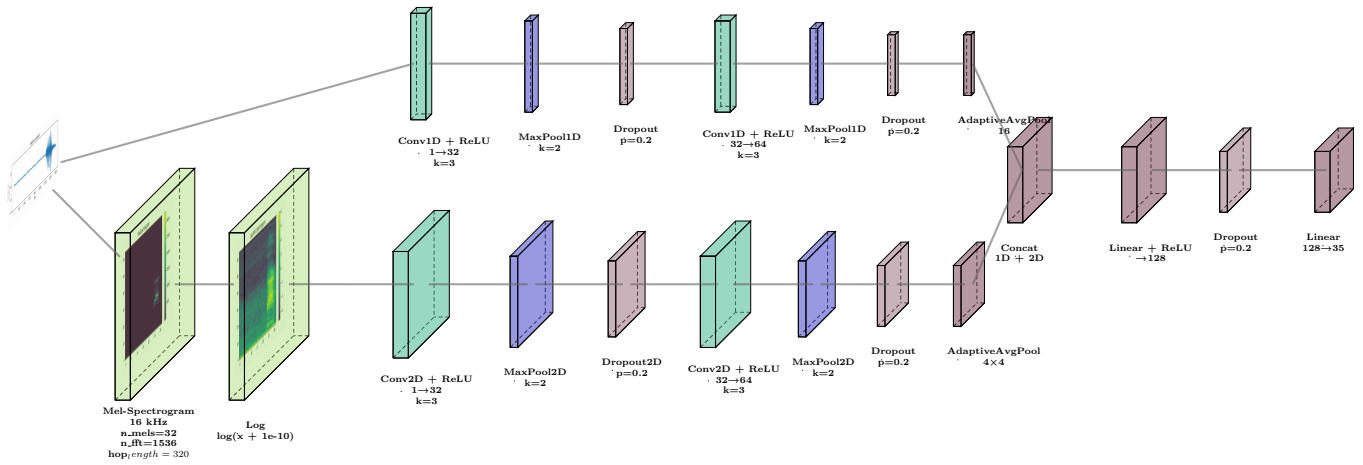
On a aussi vu quelques projet qui utilisent ce genre d'astuce, on a souhaiter voir si cela était pertinent dans notre cas.

Définition

Listing 12: Résultat optuna

```
1 Meilleur parametres : {'batch_size': 32, 'lr': 0.00012765384089860127, 'weight_decay':
  0.00011068942908456975, 'n_mels': 32, 'n_fft': 1536, 'hop_length': 320, 'cnn1d_out_channels_1':
  32, 'cnn1d_out_channels_2': 64, 'kernel_size_1d': 7, 'pooling_kernel_1d': 4, '
  adaptive_pool_1d_output': 12, 'cnn2d_out_channels_1': 128, 'cnn2d_out_channels_2': 224, '
  kernel_size_2d': 7, 'pooling_kernel_2d': 3, 'adaptive_pool_2d_output': (8, 8), 'fc_hidden_size':
  320, 'dropout': 0.30000000000000004}
2 Meilleur accuracy : 0.8997094479511071
```

On se retrouve avec l'architecture suivante :



Architecture pour le CNN1D2D

Pour voir l'implémentation se référer à l'annexe

Comparaisons des résultats

Évaluons les performances des différents modèles. Dans un premier temps, chaque architecture a été générée et entraînée sur 20 époques. Les métriques retenues pour l'évaluation sont l'accuracy, le F1-Score et la loss, calculées sur le jeu de données de test. Par ailleurs, nous avons optimisé les hyperparamètres de chaque modèle à l'aide de Optuna, en fixant un learning rate et un weight decay spécifiques pour chaque architecture.

Les temps d'optimisation (Optuna), d'entraînement et de test ont également été mesurés afin d'évaluer l'efficacité de chaque approche. Les résultats obtenus sont présentés dans les tableaux ci-dessous. Le premier tableau à gauche compare les performances des modèles en termes de métriques classiques, tandis que le tableau de droite détaille les hyperparamètres utilisés pour l'optimiseur Adam. Enfin, le dernier tableau résume les temps de calcul associés à chaque étape.

Comparaison des performances sur Speech Commands.

Modèle	Accuracy (%)	F1-Score	Loss
CNN	92.28	0.9227	0.2795
BiLSTM	92.86	0.9289	0.2355
CNN1D2D	85.03	0.8497	0.4940
CNNTrad	87.10	0.8714	0.4562
CNNOne	83.56	0.8360	0.6387

Comparaison des temps sur Speech Commands.

Modèle	Temps optuna	Temps train (min)	Nombre époques	Temps test (s)
CNN	1h13	8.6	19	4.55
BiLSTM	1h23	3.71	11	5.06
CNN1D2D	1h59	9.23	20	5.07
CNNTrad	1h22	5.21	11	4.66
CNNOne	1h05	5.25	20	4.63

Paramètres de l'entraînement

Modèle	Learning Rate	Weight Decay	Batch Size
CNN	0.00010012603121343938	1.6319322258025493e-07	16
BiLSTM	0.0010061137930597018	1.1559805487396425e-06	64
CNN1D2D	0.00012765384089860127	0.00011068942908456975	32
CNNTrad	0.00021495551281313965	7.109975589372269e-07	16
CNNOne	0.0006425519426416211	2.929655627975144e-05	32

Augmentation des données

Résultats / Impact

Nouvelles données

Nos échantillons

Nous proposons la possibilité de tester le modèle sur un enregistrement en direct.

Listing 13: Exemple d'utilisation de la fonction

```
1 import sounddevice as sd
2 from scipy.io.wavfile import write
3
4 """
5 print(sd.query_devices())
6 device_info = sd.query_devices(10)
7 print(device_info)
8 """
9
10 model = ...()
11 model.load_state_dict(torch.load(f"best_model_{model.__class__.__name__}.pth", weights_only=True))
12 device_index = 10
13
14 sd.default.device = device_index
15
16 duration = 1
17 fs = int(sd.query_devices(device_index)["default_samplerate"])
18 filename = "capture.wav"
19
20 print("Enregistrement en cours...")
21 audio = sd.rec(int(duration * fs), samplerate=fs, channels=1)
22 sd.wait()
23 print("Enregistrement termine.")
24
25 audio = torch.from_numpy(audio.T).float()
26
27 if fs != 16000 :
28     resampler = torchaudio.transforms.Resample(orig_freq=fs, new_freq=16000)
29     audio = resampler(audio)
30
31
32 model.eval()
33
34 softmax_eval_live = torch.nn.Softmax(dim=1)
35
36 plt.figure(figsize=(20,5))
37 plt.xticks(rotation=90)
38 plt.title("Distribution")
39 plt.bar(labels, softmax_eval_live(model(audio))[0].detach().numpy())
```


Bonus : GAN

Bonus : Reconnaissance des locuteurs

Conclusion

Annexes

Listing 14: Optuna pipeline optunatraining.py

```

1 def objective(trial, modelClass):
2     batch_size = trial.suggest_categorical("batch_size", [16, 32, 64])
3     lr = trial.suggest_float("lr", 1e-5, 1e-2, log=True)
4
5     #Decrit comme une regularisation pour l'optimizer
6     weight_decay = trial.suggest_float("weight_decay", 1e-7, 1e-3, log=True)
7     params = modelClass.optimize(trial)
8
9     # Initialisation du modele
10    model = modelClass(params).to(device)
11    train_loader, val_loader = get_loaders(batch_size) #Permet de creer train_loader et val_loader
12    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
13    criterion = torch.nn.CrossEntropyLoss()
14    best_acc = 0
15    patience = 2
16    wait = 0
17    for epoch in range(3):
18
19        model.train()
20        for x, y in train_loader:
21            x, y = x.to(device), y.to(device)
22
23            optimizer.zero_grad()
24            loss = criterion(model(x), y)
25            loss.backward()
26            optimizer.step()
27
28        model.eval()
29        correct = 0
30        total = 0
31
32        with torch.no_grad():
33            for x, y in val_loader:
34                x, y = x.to(device), y.to(device)
35                preds = model(x).argmax(1)
36
37                correct += (preds == y).sum().item()
38                total += y.size(0)
39
40        acc = correct / total
41
42        trial.report(acc, epoch)
43
44        #Permet de faire un elagage
45        if trial.should_prune():
46            raise optuna.TrialPruned()
47
48        #EarlyStopping
49        if acc > best_acc:
50            best_acc = acc
51            wait = 0
52        else:
53            wait += 1
54            if wait >= patience:
55                break
56
57    return best_acc
58
59 #Ici on import le modele
60 from ... import ...
61
62 if __name__ == "__main__":
63     #Permet le multi-processing
64     mp.set_start_method("spawn", force=True)
65
66     study = optuna.create_study(direction="maximize")
67     study.optimize(partial(objective, modelClass=...), n_trials=50)
68
69     print("Meilleur parametres :", study.best_params)
70     print("Meilleur accuracy :", study.best_value)

```

Listing 15: Méthode pour l'entraînement d'un modèle training_CNNModule.py

```

1 import torchaudio
2 from torch.utils.data import DataLoader
3 import torch
4 import torch.optim as optim
5 import torchaudio.transforms as T
6 import torch.multiprocessing as mp
7 import time
8 import torch.nn as nn
9
10 data = torchaudio.datasets.SPEECHCOMMANDS(download=True, root=".")
11 data_train = torchaudio.datasets.SPEECHCOMMANDS(download=True, root=".", subset="training")
12 data_validation = torchaudio.datasets.SPEECHCOMMANDS(download=True, root=".", subset="validation")
13 data_testing = torchaudio.datasets.SPEECHCOMMANDS(download=True, root=".", subset="testing")
14 labels = ['backward', 'bed', 'bird', 'cat', 'dog', 'down', 'eight', 'five', 'follow', 'forward', 'four', 'go', 'happy', 'house', 'learn', 'left', 'marvin', 'nine', 'no', 'off', 'on', 'one', 'right', 'seven', 'sheila', 'six', 'stop', 'three', 'tree', 'two', 'up', 'visual', 'wow', 'yes', 'zero']
15
16
17 label_to_idx = {l: i for i, l in enumerate(labels)}
18
19 def pre_process_batch(batch):
20     target_len = 16000
21     new_batch = []
22
23     for waveform, sample_rate, label_str, speaker_id, utterance_number in batch:
24
25         current_len = waveform.shape[1]
26         if current_len < target_len:
27             waveform = torch.nn.functional.pad(waveform, (0, target_len - current_len))
28
29         label = label_to_idx.get(label_str, None)
30         if label is None:
31             continue
32
33         new_batch.append((waveform, label))
34
35     waveforms = torch.stack([item[0] for item in new_batch])
36     labels_tensor = torch.tensor([item[1] for item in new_batch])
37
38     return waveforms, labels_tensor
39
40 from ... import ...
41 lr = ...
42 wd = ...
43 batch_size = ...
44
45 train_loader = DataLoader(
46     data_train,
47     batch_size=batch_size,
48     shuffle=True,
49     num_workers=8,
50     pin_memory=True,
51     persistent_workers=True,
52     collate_fn=pre_process_batch
53 )
54
55 val_loader = DataLoader(
56     data_validation,
57     batch_size=batch_size,
58     shuffle=True,
59     num_workers=8,
60     pin_memory=True,
61     persistent_workers=True,
62     collate_fn=pre_process_batch
63 )
64
65 ##Boucle d'entraînement
66 if __name__ == "__main__":
67     ##Permet de faire du parallelisme
68     mp.set_start_method("spawn", force=True)
69     device = "cuda"
70
71     start_time = time.perf_counter()
72
73     model = ...()
74     model.to("cuda")
75

```

```

76 criterion = torch.nn.CrossEntropyLoss()
77 optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=wd)
78 model.train()
79
80 epochs = 20
81 best_loss = float('inf')
82 patience = 5
83 trigger_times = 0
84
85 for epoch in range(epochs):
86     model.train()
87     running_loss = 0.0
88
89     for batch_idx, (waveforms, labels_batch) in enumerate(train_loader):
90         waveforms = waveforms.to(device)
91         labels_batch = labels_batch.to(device)
92
93         optimizer.zero_grad()
94         outputs = model(waveforms)
95         loss = criterion(outputs, labels_batch)
96         loss.backward()
97         optimizer.step()
98
99         running_loss += loss.item()
100
101         if batch_idx % 50 == 0:
102             print(f"Epoch {epoch+1}/{epochs} | Batch {batch_idx}/{len(train_loader)} | Loss: {
103                 loss.item():.4f}")
104
105     train_loss = running_loss / len(train_loader)
106
107     model.eval()
108     val_loss = 0.0
109     correct = 0
110     total = 0
111     with torch.no_grad():
112         for waveforms, labels_batch in val_loader:
113             waveforms = waveforms.to(device)
114             labels_batch = labels_batch.to(device)
115             outputs = model(waveforms)
116             loss = criterion(outputs, labels_batch)
117             val_loss += loss.item()
118             _, predicted = torch.max(outputs, 1)
119             total += labels_batch.size(0)
120             correct += (predicted == labels_batch).sum().item()
121
122     val_loss /= len(val_loader)
123     val_acc = correct / total
124
125     print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f} |
126         Val Acc: {val_acc:.4f}")
127
128     #EarlyStopping
129     if val_loss < best_loss:
130         best_loss = val_loss
131         trigger_times = 0
132         torch.save(model.state_dict(), f'best_model_{model.__class__.__name__}.pth')
133         print(f"Nouveau meilleur modele sauvegarde (Validation Loss: {best_loss:.4f})")
134     else:
135         trigger_times += 1
136         print(f"Pas d'amelioration.")
137         if trigger_times >= patience:
138             print(f"Early Stopping declenche. Epoque : {epoch+1}")
139             break
140
141 end_time = time.perf_counter()
142 total_time = end_time - start_time
143 print(f"\nTemps total d'entrainement : {total_time:.2f} secondes")
144 exit()

```

Listing 16: Code pour le test sur le dataset test_CNNModule.py

```

1 import torch
2 import torchaudio
3 from torch.utils.data import DataLoader
4 from sklearn.metrics import f1_score, accuracy_score
5 import numpy as np
6 import time
7
8 start_time = time.time()
9
10
11 data_testing = torchaudio.datasets.SPEECHCOMMANDS(download=True, root=".", subset="testing")
12 labels = ['backward', 'bed', 'bird', 'cat', 'dog', 'down', 'eight', 'five', 'follow', 'forward', 'four', 'go', 'happy', 'house', 'learn', 'left', 'marvin', 'nine', 'no', 'off', 'on', 'one', 'right', 'seven', 'sheila', 'six', 'stop', 'three', 'tree', 'two', 'up', 'visual', 'wow', 'yes', 'zero']
13
14
15 label_to_idx = {l: i for i, l in enumerate(labels)}
16
17 def pre_process_batch(batch):
18     target_len = 16000
19     new_batch = []
20
21     for waveform, sample_rate, label_str, speaker_id, utterance_number in batch:
22
23         current_len = waveform.shape[1]
24         if current_len < target_len:
25             waveform = torch.nn.functional.pad(waveform, (0, target_len - current_len))
26
27         label = label_to_idx.get(label_str, None)
28         if label is None:
29             continue
30
31         new_batch.append((waveform, label))
32
33     waveforms = torch.stack([item[0] for item in new_batch])
34     labels_tensor = torch.tensor([item[1] for item in new_batch])
35
36     return waveforms, labels_tensor
37
38 test_loader = DataLoader(
39     data_testing,
40     batch_size=32,
41     shuffle=False,
42     collate_fn=pre_process_batch
43 )
44
45 from ... import ...
46
47 device = "cuda"
48 model = ...()
49 model.to(device)
50 model.load_state_dict(torch.load(f"best_model_{model.__class__.__name__}.pth", weights_only=True))
51 model.eval()
52
53 criterion = torch.nn.CrossEntropyLoss()
54 all_labels = []
55 all_predictions = []
56 test_loss = 0.0
57
58 with torch.no_grad():
59     for waveforms, labels_batch in test_loader:
60         waveforms = waveforms.to(device)
61         labels_batch = labels_batch.to(device)
62
63         outputs = model(waveforms)
64         loss = criterion(outputs, labels_batch)
65         test_loss += loss.item()
66
67         _, predicted = torch.max(outputs, 1)
68
69         all_labels.extend(labels_batch.cpu().numpy())
70         all_predictions.extend(predicted.cpu().numpy())
71
72
73 test_loss /= len(test_loader)
74 accuracy = accuracy_score(all_labels, all_predictions)
75 f1 = f1_score(all_labels, all_predictions, average="weighted")

```

```
76|
77| print("\n--- M triques sur le jeu de test ---")
78| print(f"Test Loss: {test_loss:.4f}")
79| print(f"Accuracy: {accuracy * 100:.2f}%")
80| print(f"F1-Score (weighted): {f1:.4f}")
81|
82| end_time = time.time()
83| elapsed_time = end_time - start_time
84|
85| print("\n--- Temps de calcul ---")
86| print(f"Temps    coul    : {elapsed_time:.2f} secondes")
```

Listing 17: Implémentation du modèle CNN cnn.py

```

1 class CNNModule(torch.nn.Module):
2     def __init__(self, params=None):
3         super().__init__()
4
5         if params is None:
6             params = self.optimize_defaults()
7
8         self.spec = T.MelSpectrogram(
9             sample_rate=16000,
10            n_fft=params["n_fft"],
11            hop_length=params["hop_length"],
12            n_mels=params["n_mels"]
13        )
14        self.cnn = torch.nn.Sequential(
15            torch.nn.Conv2d(1, params["cnn1_out_channels"], kernel_size=params["kernel_size"],
16                           padding=params["kernel_size"] // 2),
17            torch.nn.ReLU(),
18            torch.nn.MaxPool2d(params["pooling_kernel"]),
19            torch.nn.Dropout2d(params["dropout"]),
20            torch.nn.Conv2d(params["cnn1_out_channels"], params["cnn2_out_channels"], kernel_size=
21                           params["kernel_size"], padding=params["kernel_size"] // 2),
22            torch.nn.ReLU(),
23            torch.nn.MaxPool2d(params["pooling_kernel"]),
24            torch.nn.Dropout2d(params["dropout"])
25        )
26
27        self.pool = torch.nn.AdaptiveAvgPool2d(params["adaptive_pool_output"])
28
29        self.flatten = torch.nn.Flatten()
30
31        pooled_height, pooled_width = params["adaptive_pool_output"]
32
33        self.dense = torch.nn.Sequential(
34            torch.nn.Linear(params["cnn2_out_channels"] * pooled_height * pooled_width, 128),
35            torch.nn.ReLU(),
36            torch.nn.Dropout(params["dropout"]),
37            torch.nn.Linear(128, params["fc_output_size"])
38        )
39
40    def forward(self, x):
41        x = self.spec(x)
42        x = torch.log(x + 1e-10)
43
44        #Permet de passer un seul element pendant la phase de production
45        if len(x.shape) < 3:
46            x = x.unsqueeze(0)
47
48        x = self.cnn(x)
49
50        x = self.pool(x)
51        x = self.flatten(x)
52        return self.dense(x)
53
54    @staticmethod
55    def optimize(trial):
56        #Dans l'idée c'est pour optimiser le pipeline de optuna
57        params = {
58            "n_mels": trial.suggest_int("n_mels", 32, 128, step=16),
59            "n_fft": trial.suggest_int("n_fft", 512, 2048, step=256),
60            "hop_length": trial.suggest_int("hop_length", 256, 1024, step=64),
61
62            "cnn1_out_channels": trial.suggest_int("cnn1_out_channels", 16, 128, step=16),
63            "cnn2_out_channels": trial.suggest_int("cnn2_out_channels", 32, 256, step=16),
64            "kernel_size": trial.suggest_int("kernel_size", 3, 7, step=2),
65            "pooling_kernel": trial.suggest_int("pooling_kernel", 2, 4),
66
67            "adaptive_pool_output": trial.suggest_categorical(
68                "adaptive_pool_output",
69                [(4, 4), (8, 8), (2, 2)]
70            ),
71
72            "fc_output_size": 35,
73            "dropout": trial.suggest_float("dropout", 0.0, 0.5, step=0.1),
74        }
75        return params
76
77    @staticmethod

```



```
77 def optimize_defaults():
78     #Ces parametres ont etait determinees avec optuna
79     return {
80         "n_mels": 48,
81         "n_fft": 1536,
82         "hop_length": 384,
83
84         "cnn1_out_channels": 48,
85         "cnn2_out_channels": 208,
86         "kernel_size": 7,
87         "pooling_kernel": 2,
88
89         "adaptive_pool_output": (8, 8),
90
91         "fc_output_size": 35,
92         "dropout": 0.2,
93     }
```

Listing 18: Définition du Modèle CNN2D1D `cnn_1d_2d.py`

```

1 import torch
2 import torchaudio.transforms as T
3
4 class DoubleCNNModule(torch.nn.Module):
5
6     def __init__(self, params=None):
7         super().__init__()
8
9         if params is None:
10             params = self.optimize_defaults()
11
12         self.spec = T.MelSpectrogram(
13             sample_rate=16000,
14             n_fft=params["n_fft"],
15             hop_length=params["hop_length"],
16             n_mels=params["n_mels"]
17         )
18
19         self.cnn1D = torch.nn.Sequential(
20             torch.nn.Conv1d(
21                 in_channels=1,
22                 out_channels=params["cnn1d_out_channels_1"],
23                 kernel_size=params["kernel_size_1d"],
24                 padding=params["kernel_size_1d"] // 2
25             ),
26             torch.nn.ReLU(),
27             torch.nn.MaxPool1d(params["pooling_kernel_1d"]),
28             torch.nn.Dropout(params["dropout"]),
29
30             torch.nn.Conv1d(
31                 in_channels=params["cnn1d_out_channels_1"],
32                 out_channels=params["cnn1d_out_channels_2"],
33                 kernel_size=params["kernel_size_1d"],
34                 padding=params["kernel_size_1d"] // 2
35             ),
36             torch.nn.ReLU(),
37             torch.nn.MaxPool1d(params["pooling_kernel_1d"]),
38             torch.nn.Dropout(params["dropout"])
39         )
40         self.pool1D = torch.nn.AdaptiveAvgPool1d(params["adaptive_pool_1d_output"])
41
42         self.cnn2D = torch.nn.Sequential(
43             torch.nn.Conv2d(
44                 in_channels=1,
45                 out_channels=params["cnn2d_out_channels_1"],
46                 kernel_size=params["kernel_size_2d"],
47                 padding=params["kernel_size_2d"] // 2
48             ),
49             torch.nn.ReLU(),
50             torch.nn.MaxPool2d(params["pooling_kernel_2d"]),
51             torch.nn.Dropout2d(params["dropout"]),
52
53             torch.nn.Conv2d(
54                 in_channels=params["cnn2d_out_channels_1"],
55                 out_channels=params["cnn2d_out_channels_2"],
56                 kernel_size=params["kernel_size_2d"],
57                 padding=params["kernel_size_2d"] // 2
58             ),
59             torch.nn.ReLU(),
60             torch.nn.MaxPool2d(params["pooling_kernel_2d"]),
61             torch.nn.Dropout2d(params["dropout"])
62         )
63         self.pool2D = torch.nn.AdaptiveAvgPool2d(params["adaptive_pool_2d_output"])
64
65         self.flatten = torch.nn.Flatten()
66
67         pooled_1d_size = params["cnn1d_out_channels_2"] * params["adaptive_pool_1d_output"]
68         pooled_2d_height, pooled_2d_width = params["adaptive_pool_2d_output"]
69         pooled_2d_size = params["cnn2d_out_channels_2"] * pooled_2d_height * pooled_2d_width
70
71         total_fc_input = pooled_1d_size + pooled_2d_size
72
73         self.dense = torch.nn.Sequential(
74             torch.nn.Linear(total_fc_input, params["fc_hidden_size"]),
75             torch.nn.ReLU(),
76             torch.nn.Dropout(params["dropout"]),
77             torch.nn.Linear(params["fc_hidden_size"], params["fc_output_size"])
78         )

```

```

79
80 def forward(self, x):
81
82     x_1D = self.cnn1D(x)
83     x_1D = self.pool1D(x_1D)
84     x_1D = self.flatten(x_1D)
85
86     x_2D = self.spec(x)
87     x_2D = torch.log(x_2D + 1e-10)
88     if x_2D.dim() == 3:
89         x_2D = x_2D.unsqueeze(0)
90     x_2D = self.cnn2D(x_2D)
91     x_2D = self.pool2D(x_2D)
92     x_2D = self.flatten(x_2D)
93
94     x = torch.cat([x_1D, x_2D], dim=1)
95
96     return self.dense(x)
97
98 @staticmethod
99 def optimize(trial):
100     params = {
101
102         "n_mels": trial.suggest_int("n_mels", 32, 128, step=16),
103         "n_fft": trial.suggest_int("n_fft", 512, 2048, step=256),
104         "hop_length": trial.suggest_int("hop_length", 256, 1024, step=64),
105
106         "cnn1d_out_channels_1": trial.suggest_int("cnn1d_out_channels_1", 16, 128, step=16),
107         "cnn1d_out_channels_2": trial.suggest_int("cnn1d_out_channels_2", 32, 256, step=16),
108         "kernel_size_1d": trial.suggest_int("kernel_size_1d", 3, 7, step=2),
109         "pooling_kernel_1d": trial.suggest_int("pooling_kernel_1d", 2, 4),
110         "adaptive_pool_1d_output": trial.suggest_int("adaptive_pool_1d_output", 8, 32, step=4),
111
112         "cnn2d_out_channels_1": trial.suggest_int("cnn2d_out_channels_1", 16, 128, step=16),
113         "cnn2d_out_channels_2": trial.suggest_int("cnn2d_out_channels_2", 32, 256, step=16),
114         "kernel_size_2d": trial.suggest_int("kernel_size_2d", 3, 7, step=2),
115         "pooling_kernel_2d": trial.suggest_int("pooling_kernel_2d", 2, 4),
116         "adaptive_pool_2d_output": trial.suggest_categorical(
117             "adaptive_pool_2d_output", [(4, 4), (8, 8), (2, 2)]
118         ),
119
120         "fc_hidden_size": trial.suggest_int("fc_hidden_size", 64, 512, step=64),
121         "fc_output_size": 35,
122         "dropout": trial.suggest_float("dropout", 0.0, 0.5, step=0.1),
123     }
124     return params
125
126 @staticmethod
127 def optimize_defaults():
128     return {
129         "n_mels": 32,
130         "n_fft": 1536,
131         "hop_length": 320,
132         "cnn1d_out_channels_1": 32,
133         "cnn1d_out_channels_2": 64,
134         "kernel_size_1d": 3,
135         "pooling_kernel_1d": 2,
136         "adaptive_pool_1d_output": 16,
137         "cnn2d_out_channels_1": 32,
138         "cnn2d_out_channels_2": 64,
139         "kernel_size_2d": 3,
140         "pooling_kernel_2d": 2,
141         "adaptive_pool_2d_output": (4, 4),
142         "fc_hidden_size": 128,
143         "fc_output_size": 35,
144         "dropout": 0.2,
145     }

```

Listing 19: Définition du Modèle CNNBiLSTM cnn_bilstm.py

```

1 class CNNBiLSTM(torch.nn.Module):
2     def __init__(self, params=None):
3         super().__init__()
4
5         if params is None:
6             params = self.optimize_defaults()
7
8         self.spec = T.MelSpectrogram(
9             sample_rate=16000,
10            n_fft=params["n_fft"],
11            hop_length=params["hop_length"],
12            n_mels=params["n_mels"]
13        )
14
15        self.cnn = torch.nn.Sequential(
16            torch.nn.Conv2d(
17                in_channels=1,
18                out_channels=params["cnn1_out_channels"],
19                kernel_size=params["kernel_size"],
20                padding=params["kernel_size"] // 2
21            ),
22            torch.nn.ReLU(),
23            torch.nn.MaxPool2d(params["pooling_kernel"]),
24            torch.nn.Dropout2d(params["dropout"]),
25
26            torch.nn.Conv2d(
27                in_channels=params["cnn1_out_channels"],
28                out_channels=params["cnn2_out_channels"],
29                kernel_size=params["kernel_size"],
30                padding=params["kernel_size"] // 2
31            ),
32            torch.nn.ReLU(),
33            torch.nn.MaxPool2d(params["pooling_kernel"]),
34            torch.nn.Dropout2d(params["dropout"])
35        )
36
37        self.adaptive_pool = torch.nn.AdaptiveAvgPool2d(params["adaptive_pool_output"])
38
39        pooled_height, pooled_width = params["adaptive_pool_output"]
40        cnn_output_features = params["cnn2_out_channels"] * pooled_width
41        self.lstm = torch.nn.LSTM(
42            input_size=cnn_output_features,
43            hidden_size=params["lstm_hidden_size"],
44            num_layers=params["lstm_num_layers"],
45            batch_first=True,
46            bidirectional=True,
47            dropout=params["dropout"] if params["lstm_num_layers"] > 1 else 0.0
48        )
49
50        self.fc = torch.nn.Sequential(
51            torch.nn.Linear(params["lstm_hidden_size"] * 2, params["fc_hidden_size"]),
52            torch.nn.ReLU(),
53            torch.nn.Dropout(params["dropout"]),
54            torch.nn.Linear(params["fc_hidden_size"], params["fc_output_size"])
55        )
56
57    def forward(self, x):
58        x = self.spec(x)
59        x = torch.log(x + 1e-10)
60
61        if x.dim() == 3:
62            x = x.unsqueeze(0)
63
64        x = self.cnn(x)
65        x = self.adaptive_pool(x)
66
67        batch_size = x.size(0)
68        x = x.permute(0, 3, 1, 2)
69        x = x.reshape(batch_size, x.size(1), -1)
70
71        x, (h_n, c_n) = self.lstm(x)
72
73        forward_hidden = h_n[-2]
74        backward_hidden = h_n[-1]
75        x = torch.cat([forward_hidden, backward_hidden], dim=1)
76
77        return self.fc(x)
78

```

```

79 | @staticmethod
80 | def optimize(trial):
81 |     params = {
82 |         "n_mels": trial.suggest_int("n_mels", 32, 128, step=16),
83 |         "n_fft": trial.suggest_int("n_fft", 512, 2048, step=256),
84 |         "hop_length": trial.suggest_int("hop_length", 256, 1024, step=64),
85 |
86 |         "cnn1_out_channels": trial.suggest_int("cnn1_out_channels", 16, 128, step=16),
87 |         "cnn2_out_channels": trial.suggest_int("cnn2_out_channels", 32, 256, step=16),
88 |         "kernel_size": trial.suggest_int("kernel_size", 3, 7, step=2),
89 |         "pooling_kernel": trial.suggest_int("pooling_kernel", 2, 4),
90 |         "adaptive_pool_output": trial.suggest_categorical(
91 |             "adaptive_pool_output", [(4, 4), (8, 8), (2, 2)]
92 |         ),
93 |
94 |         "lstm_hidden_size": trial.suggest_int("lstm_hidden_size", 64, 256, step=32),
95 |         "lstm_num_layers": trial.suggest_int("lstm_num_layers", 1, 3),
96 |
97 |         "fc_hidden_size": trial.suggest_int("fc_hidden_size", 64, 512, step=64),
98 |         "fc_output_size": 35,
99 |         "dropout": trial.suggest_float("dropout", 0.0, 0.5, step=0.1),
100 |     }
101 |     return params
102 |
103 | @staticmethod
104 | def optimize_defaults():
105 |     return {
106 |         "n_mels": 64,
107 |         "n_fft": 1024,
108 |         "hop_length": 256,
109 |
110 |         "cnn1_out_channels": 32,
111 |         "cnn2_out_channels": 96,
112 |         "kernel_size": 7,
113 |         "pooling_kernel": 2,
114 |         "adaptive_pool_output": (8, 8),
115 |
116 |         "lstm_hidden_size": 160,
117 |         "lstm_num_layers": 2,
118 |         "fc_hidden_size": 384,
119 |         "fc_output_size": 35,
120 |         "dropout": 0.1,
121 |     }

```

Listing 20: Définition du Modèle CNNOne

```

1 class CNNOneFPool3(torch.nn.Module):
2     def __init__(self, params=None):
3         super().__init__()
4
5         if params is None:
6             params = self.optimize_defaults()
7
8         self.spec = T.MelSpectrogram(
9             sample_rate=16000,
10            n_fft=1024,
11            hop_length=256,
12            n_mels=40,
13            win_length=400
14        )
15
16
17        self.cnn = torch.nn.Sequential(
18            torch.nn.Conv2d(
19                in_channels=1,
20                out_channels=params["cnn_out_channels"],
21                kernel_size=params["kernel_size"],
22                stride=(1, 1),
23                padding=0
24            ),
25            torch.nn.ReLU(),
26            torch.nn.MaxPool2d(
27                kernel_size=params["pooling_kernel"],
28                stride=params["pooling_kernel"]
29            )
30        )
31
32        self.flatten = torch.nn.Flatten()
33        with torch.no_grad():
34            dummy_input = torch.zeros(1, 16000)
35            dummy_spec = self.spec(dummy_input)
36            dummy_spec = torch.log(dummy_spec + 1e-10)
37            dummy_spec = dummy_spec.unsqueeze(0)
38            dummy_cnn = self.cnn(dummy_spec)
39            flattened_size = dummy_cnn.numel() // dummy_cnn.shape[0]
40
41        self.dense = torch.nn.Sequential(
42            torch.nn.Linear(flattened_size, 32),
43            torch.nn.ReLU(),
44            torch.nn.Dropout(params["dropout"]),
45            torch.nn.Linear(32, 128),
46            torch.nn.ReLU(),
47            torch.nn.Dropout(params["dropout"]),
48            torch.nn.Linear(128, 128),
49            torch.nn.ReLU(),
50            torch.nn.Dropout(params["dropout"]),
51            torch.nn.Linear(128, 128),
52            torch.nn.ReLU(),
53            torch.nn.Dropout(params["dropout"]),
54            torch.nn.Linear(128, 128),
55            torch.nn.ReLU(),
56            torch.nn.Dropout(params["dropout"]),
57            torch.nn.Linear(128, params["fc_output_size"])
58        )
59
60        def forward(self, x):
61            x = self.spec(x)
62            x = torch.log(x + 1e-10)
63            if x.dim() == 3:
64                x = x.unsqueeze(0)
65            x = self.cnn(x)
66            x = self.flatten(x)
67            return self.dense(x)
68
69        @staticmethod
70        def optimize(trial):
71            params = {
72                "n_mels": 40,
73                "n_fft": 1024,
74                "hop_length": 256,
75                "cnn_out_channels": 54,
76                "kernel_size": (32, 8),
77                "pooling_kernel": (1, 3),
78                "fc_output_size": 35,

```

```
79         "dropout": trial.suggest_float("dropout", 0.0, 0.5, step=0.1),
80     }
81     return params
82
83     @staticmethod
84     def optimize_defaults():
85         return {
86             "n_mels": 40,
87             "n_fft": 1024,
88             "hop_length": 256,
89             "cnn_out_channels": 54,
90             "kernel_size": (32, 8),
91             "pooling_kernel": (1, 3),
92             "fc_output_size": 35,
93             "dropout": 0.0,
94         }
```

Listing 21: Définition du Modèle CNNTrad

```

1 class CNNTrad(torch.nn.Module):
2     def __init__(self, params=None):
3         super().__init__()
4
5         if params is None:
6             params = self.optimize_defaults()
7
8         self.spec = T.MelSpectrogram(
9             sample_rate=16000,
10            n_fft=1024,
11            hop_length=256,
12            n_mels=40,
13            win_length=400
14        )
15
16        self.cnn = torch.nn.Sequential(
17            torch.nn.Conv2d(
18                in_channels=1,
19                out_channels=54,
20                kernel_size=(20, 8),
21                stride=(1, 1),
22                padding=0
23            ),
24            torch.nn.ReLU(),
25            torch.nn.MaxPool2d(kernel_size=(1, 3), stride=(1, 3)),
26            torch.nn.Conv2d(
27                in_channels=54,
28                out_channels=64,
29                kernel_size=(10, 4),
30                stride=(1, 1),
31                padding=0
32            ),
33            torch.nn.ReLU()
34        )
35
36        self.flatten = torch.nn.Flatten()
37
38        with torch.no_grad():
39            dummy_input = torch.zeros(1, 16000)
40            dummy_spec = self.spec(dummy_input)
41            dummy_spec = torch.log(dummy_spec + 1e-10)
42            dummy_spec = dummy_spec.unsqueeze(0)
43            dummy_cnn = self.cnn(dummy_spec)
44            flattened_size = dummy_cnn.numel() // dummy_cnn.shape[0]
45
46        self.dense = torch.nn.Sequential(
47            torch.nn.Linear(flattened_size, 32),
48            torch.nn.ReLU(),
49            torch.nn.Dropout(params["dropout"]),
50            torch.nn.Linear(32, 128),
51            torch.nn.ReLU(),
52            torch.nn.Dropout(params["dropout"]),
53            torch.nn.Linear(128, 128),
54            torch.nn.ReLU(),
55            torch.nn.Dropout(params["dropout"]),
56            torch.nn.Linear(128, params["fc_output_size"])
57        )
58
59        def forward(self, x):
60            x = self.spec(x)
61            x = torch.log(x + 1e-10)
62
63            if x.dim() == 3:
64                x = x.unsqueeze(0)
65
66            x = self.cnn(x)
67            x = self.flatten(x)
68            return self.dense(x)
69
70        @staticmethod
71        def optimize(trial):
72            params = {
73                "n_mels": 40,
74                "n_fft": 1024,
75                "hop_length": 256,
76                "cnn1_out_channels": 54,
77                "cnn2_out_channels": 64,
78                "kernel_size": (20, 8),

```



```
79         "pooling_kernel": (1, 3),
80         "fc_output_size": 35,
81         "dropout": trial.suggest_float("dropout", 0.0, 0.5, step=0.1),
82     }
83     return params
84
85 @staticmethod
86 def optimize_defaults():
87     return {
88         "n_mels": 40,
89         "n_fft": 1024,
90         "hop_length": 256,
91         "cnn1_out_channels": 54,
92         "cnn2_out_channels": 64,
93         "kernel_size": (20, 8),
94         "pooling_kernel": (1, 3),
95         "adaptive_pool_output": None,
96         "fc_output_size": 35,
97         "dropout": 0.0,
98     }
```

Bibliographie

- <https://alexlenail.me/NN-SVG/LeNet.html>
- <https://www.overleaf.com/project/5ee514f1e88684000165e921>